AASD 4010
Deep Learning - I

Applied AI Solutions Developer Program

Architectures

Agenda

Neural Network Architectures
Feed Forward Fully Connected (FCNN)
Convolutional Neural Network (CNN)
Recurrent Neural Network (RNN)
Long Short Term Memory NN (LSTM)

NN Architectures

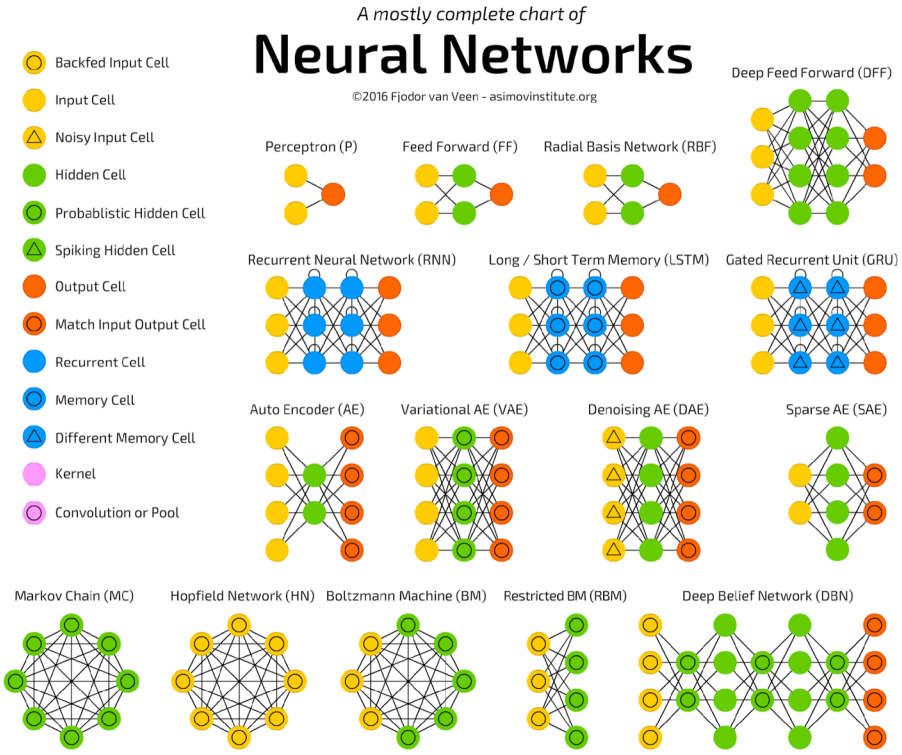
Neural Network Architecture plays a vital role in building solutions for AI problems.

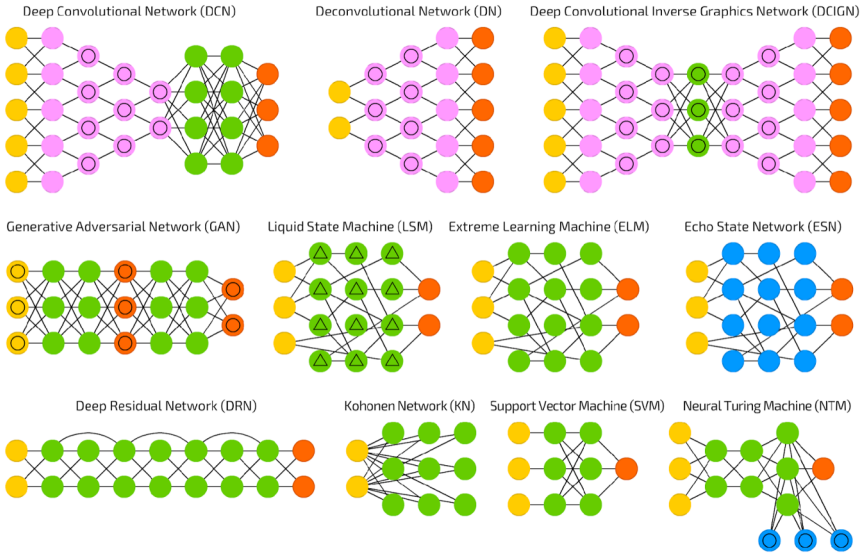
It involves careful considerations to the picking up of what architecture is best suited for what kind of problem and what kind of data.

Some examples

- Feed Forward Fully Connected Neural Network
- Convolutional Neural Network
- Recurrent Neural Network
- Long Short Term Memory Neural Network

Different structures of Neural Networks:





Problem

Problem

Problem: Text Classification

Dataset: https://www.kaggle.com/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

Fully Connected Neural Network

FCNN

FCNN

Fully Convolutional neural network is a type of network that has every neuron is connected to every other neuron in the next layer.

Steps:

Download the dataset from Kaggle
Explore the dataset
Pre-process the text
Integer encode the documents
Pad sequences to create equal-length inputs
Load GloVe embedding into memory
Create Embedding matrix
Build model
Compile model
Train the model
Evaluate the model
Plot the Accuracy and Loss

Download Glove Embeddings

Pre-trained word vectors. This data is made available under the Public Domain Dedication and License v1.0 whose full text can be found at: http://www.opendatacommons.org/licenses/pddl/1.0/.

- Wikipedia 2014 + Gigaword 5 (6B tokens, 400K vocab, uncased, 50d, 100d, 200d, & 300d vectors, 822 MB download): glove.6B.zip
- Common Crawl (42B tokens, 1.9M vocab, uncased, 300d vectors, 1.75 GB download): glove.42B.300d.zip
- Common Crawl (840B tokens, 2.2M vocab, cased, 300d vectors, 2.03 GB download): glove.840B.300d.zip
- Twitter (2B tweets, 27B tokens, 1.2M vocab, uncased, 25d, 50d, 100d, & 200d vectors, 1.42 GB download): glove.twitter.27B.zip

Integer encode the documents

```
tokenizer = Tokenizer(num_words=5000)
tokenizer.fit_on_texts(X_train)

X_train = tokenizer.texts_to_sequences(X_train)
X_test = tokenizer.texts_to_sequences(X_test)
```

Pad sequences

```
vocab_size = len(tokenizer.word_index) + 1
maxlen = 100

X_train = pad_sequences(X_train, padding='post', maxlen=maxlen)
X_test = pad_sequences(X_test, padding='post', maxlen=maxlen)
```

Load GloVe embeddings into memory

```
embeddings_dictionary = dict()
glove_file = open('glove.6B.50d.txt', encoding="utf8")

for line in glove_file:
    records = line.split()
    word = records[0]
    vector_dimensions = asarray(records[1:], dtype='float32')
    embeddings_dictionary [word] = vector_dimensions
glove_file.close()
```

Create Embedding matrix

```
embedding_matrix = zeros((vocab_size, 50))
for word, index in tokenizer.word_index.items():
    embedding_vector = embeddings_dictionary.get(word)
    if embedding_vector is not None:
        embedding_matrix[index] = embedding_vector
```

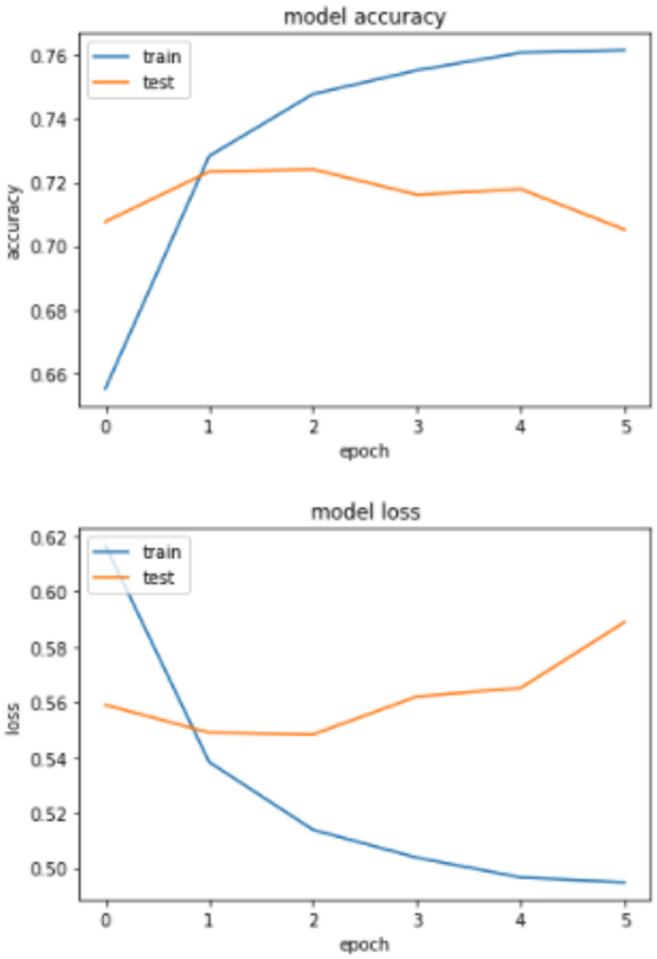
Build the model

```
model = Sequential()
embedding_layer = Embedding(vocab_size, 50,
                            weights=[embedding_matrix],
                            input_length=maxlen ,
                            trainable=False
)

model.add(embedding_layer)
model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))
```

Model Summary

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding (Embedding)        (None, 100, 50)           4627350
_________________________________________________________________
flatten (Flatten)            (None, 5000)              0
_________________________________________________________________
dense (Dense)                (None, 1)                 5001
=================================================================
Total params: 4,632,351
Trainable params: 5,001
Non-trainable params: 4,627,350
_________________________________________________________________
None
```

Training Plots



Convolutional Neural Network

CNN

CNN

Convolutional neural network is a type of network that is primarily used for 2D data classification, such as images.

For text, use 1D CNN

Idea: Find specific features in an image in the first layer.
In the next layers, the initially detected features are joined together to form bigger features.

Simple CNN for text: 1 Conv layer + 1 pooling layer

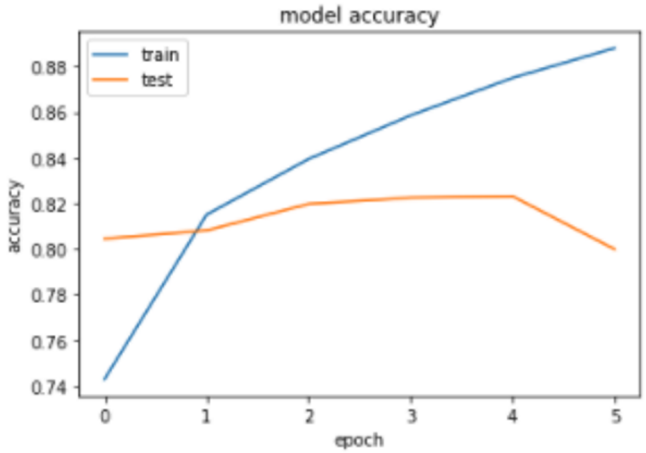
CNN

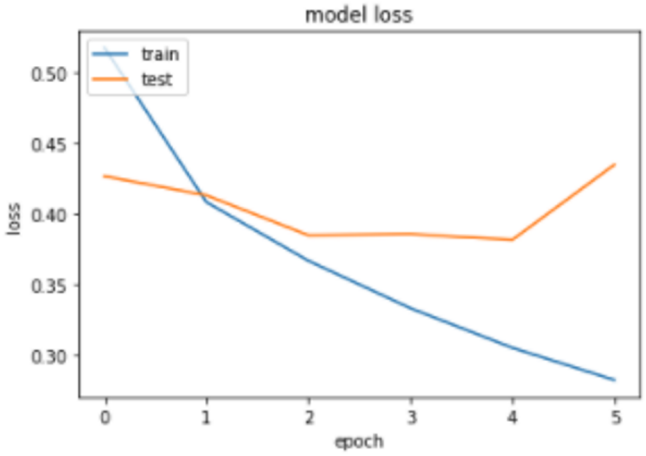
```
model = Sequential()

embedding_layer = Embedding(vocab_size,
                            50,
                            weights=[embedding_matrix],
                            input_length=maxlen,
                            trainable=False
)

model.add(embedding_layer)

model.add(Conv1D(128, 5, activation='relu'))
model.add(GlobalMaxPooling1D())
model.add(Dense(1, activation='sigmoid'))
```

CNN model summary

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding (Embedding)        (None, 100, 50)           4627350
_________________________________________________________________
conv1d (Conv1D)              (None, 96, 128)           32128
_________________________________________________________________
global_max_pooling1d (Global (None, 128)               0
_________________________________________________________________
dense (Dense)                (None, 1)                 129
=================================================================
Total params: 4,659,607
Trainable params: 32,257
Non-trainable params: 4,627,350
_________________________________________________________________
None
```

CNN Training Plots





Recurrent Neural Network

RNN

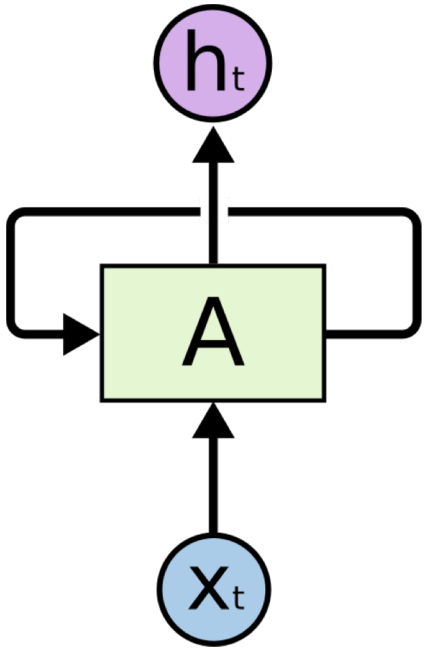
RNN

Recurrent neural network is a type of neural networks that is proven to work well with sequence data.

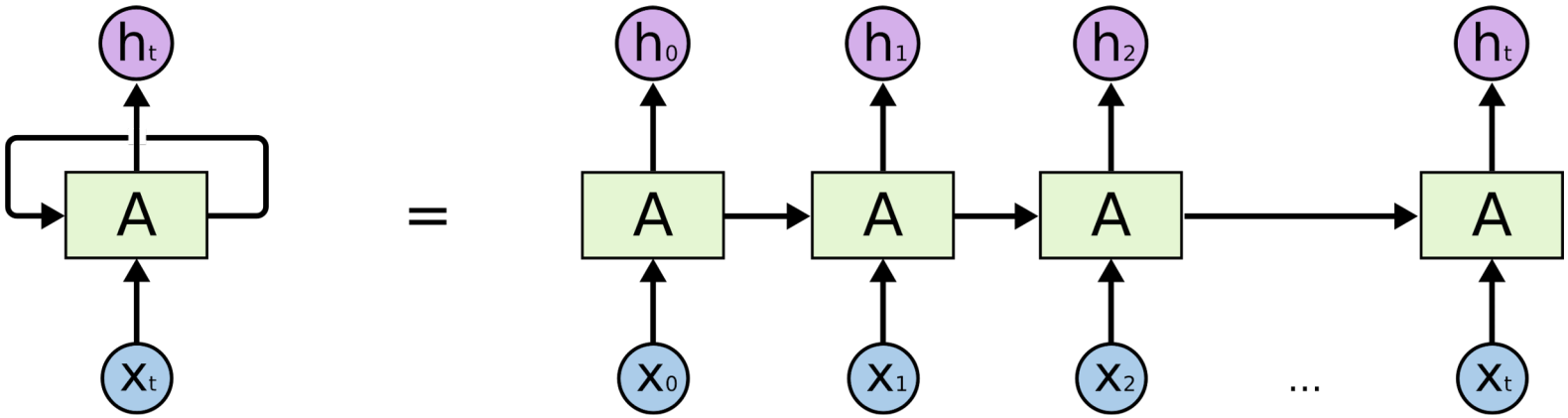
As text is a sequence of words, a recurrent neural network is an automatic choice to solve text-related problems.

Problem: Long documents or piece of text

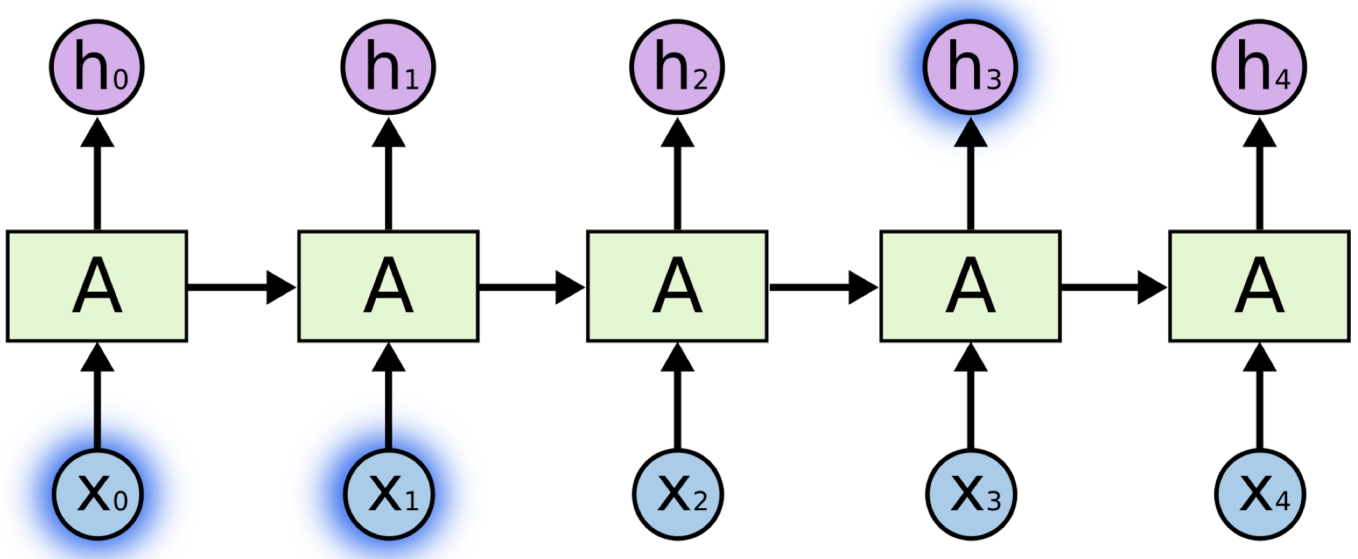
RNN



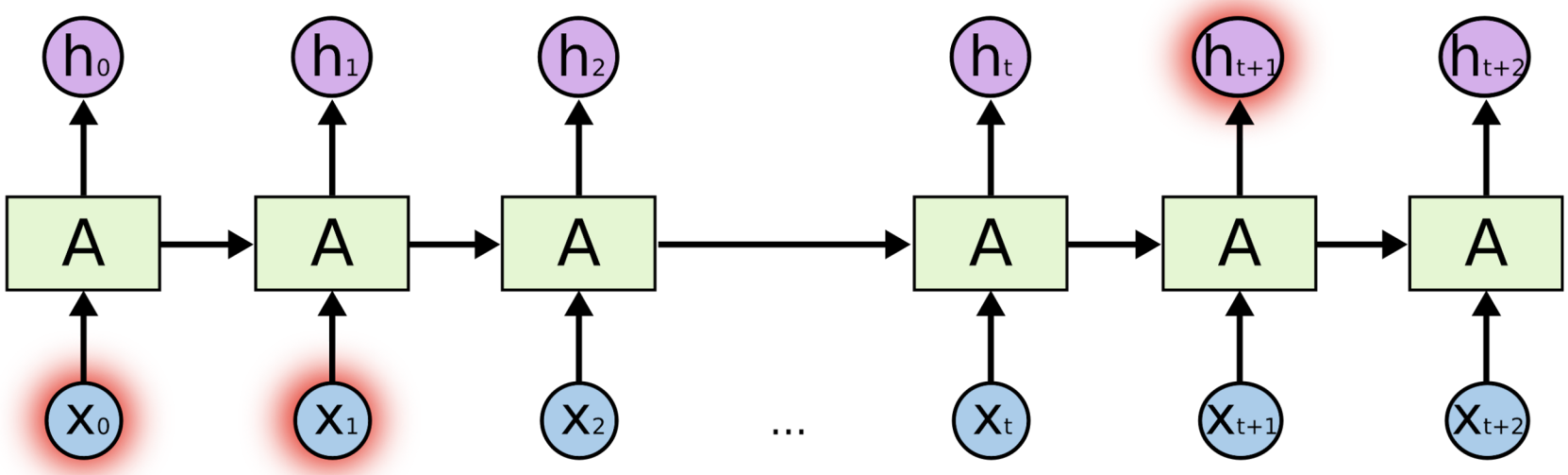
Unrolled RNN



Short term dependency



Long term dependency



Long Short Term Memory Neural Network

LSTM

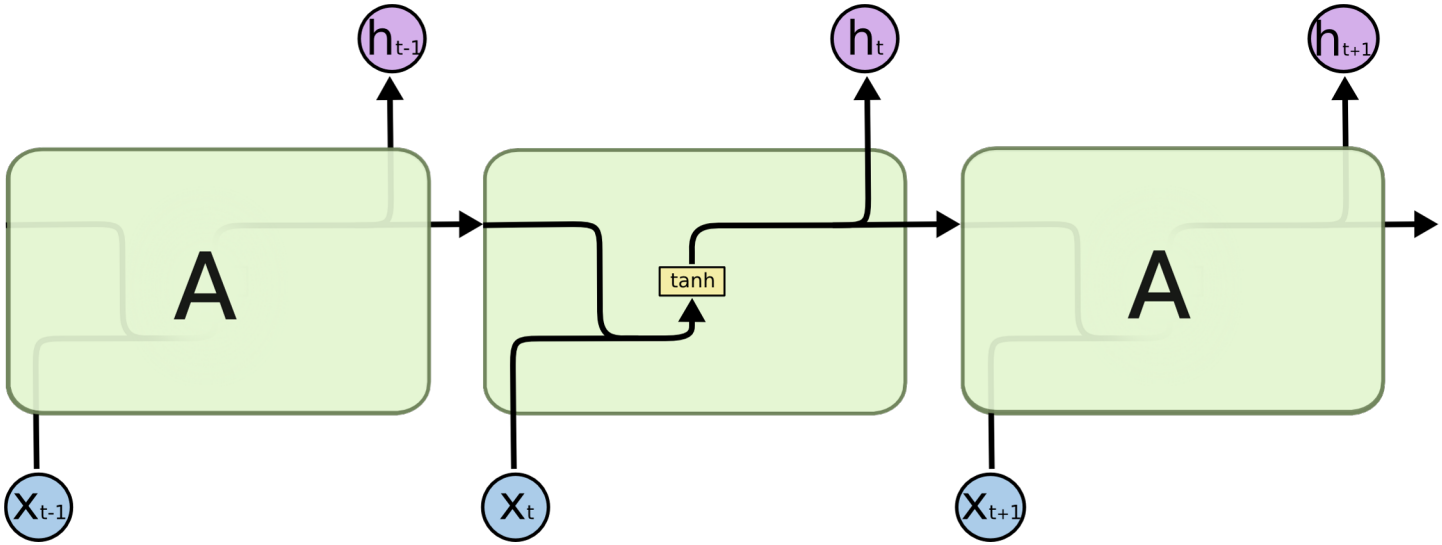
LSTM

Recurrent neural network is a type of neural networks which suffers from Short-term dependencies in its network

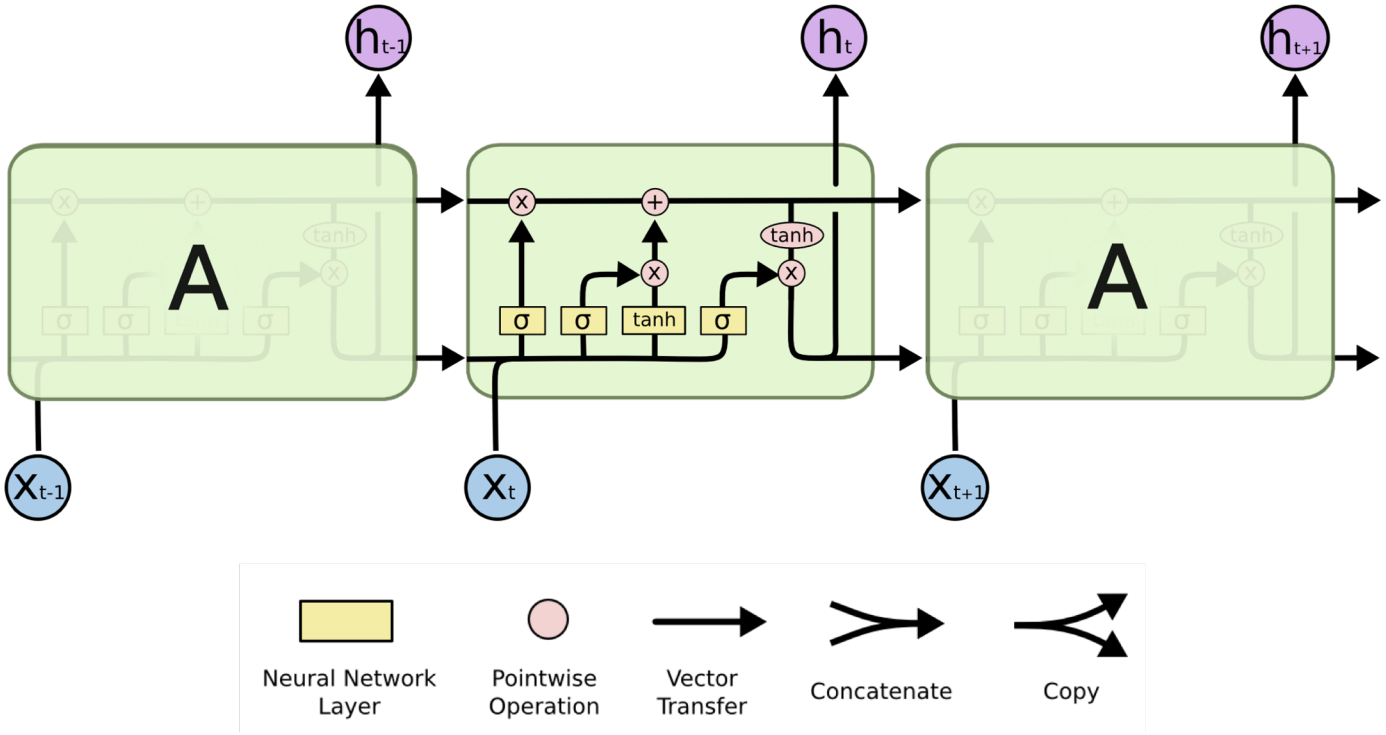
Solution: LSTM

LSTM (Long Short Term Memory network) is a variant of RNN

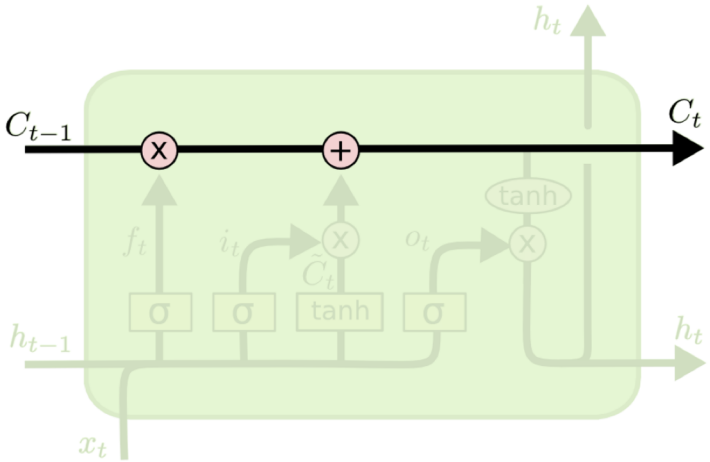
LSTM – Simple RNN

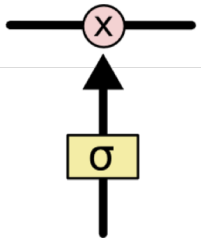


LSTM Chain


h_{t-1}
h_t
h_{t+1}
A
A
tanh
σ
σ
tanh
σ
X_{t-1}
X_t
X_{t+1}
Neural Network Layer
Pointwise Operation
Vector Transfer
Concatenate
Copy

Cell State





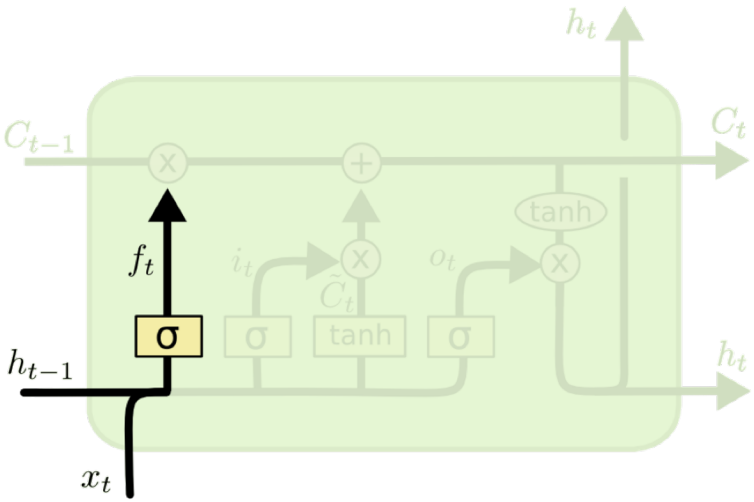
Gates

Forget Gate: What information to throw away from the cell state

Input Gate: What new information to store in the cell state

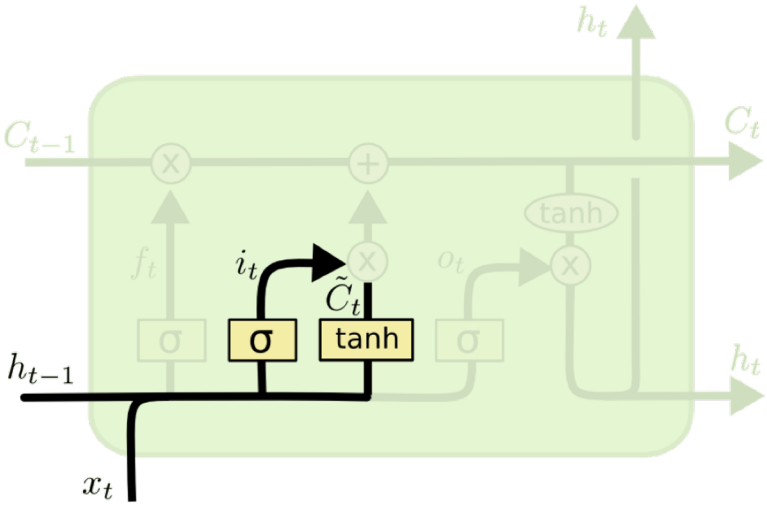
Output Gate: What information to output

Forget Gate



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

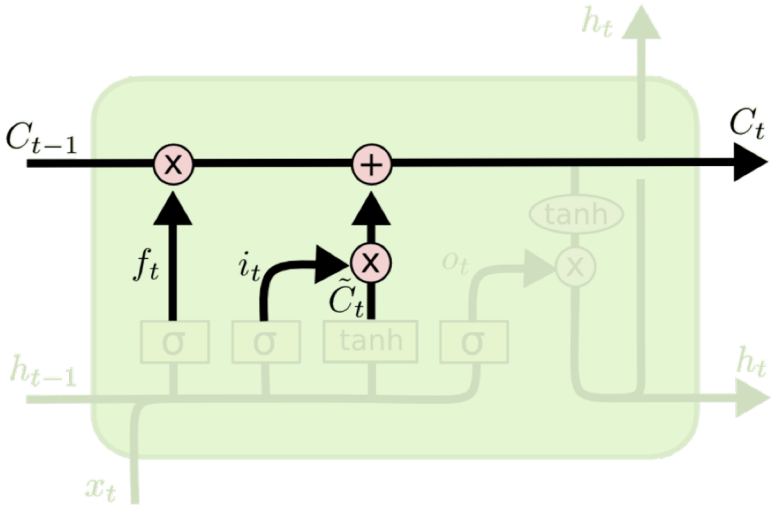
Input Gate



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

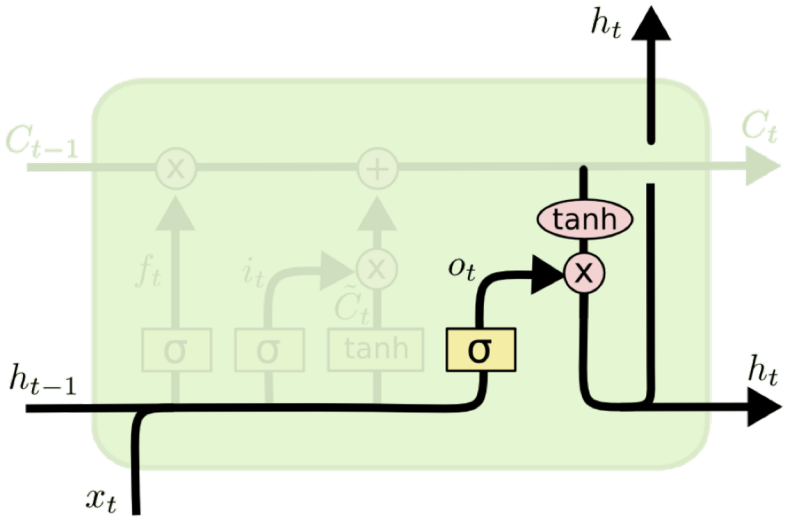
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Input Gate



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate



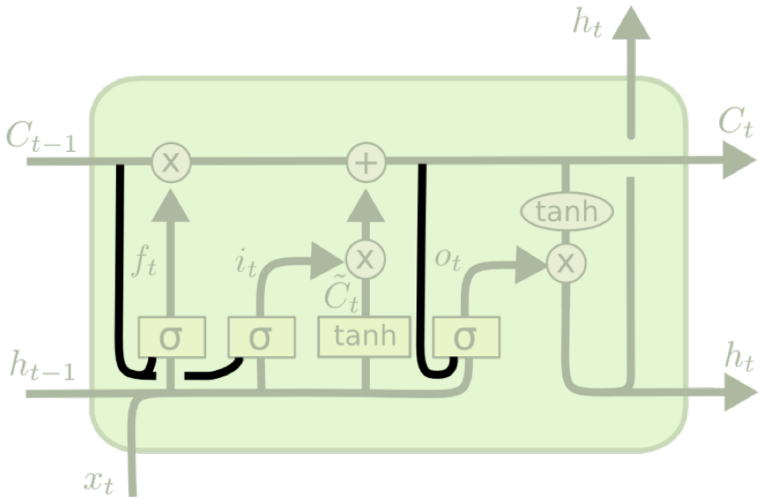
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Variations of LSTM

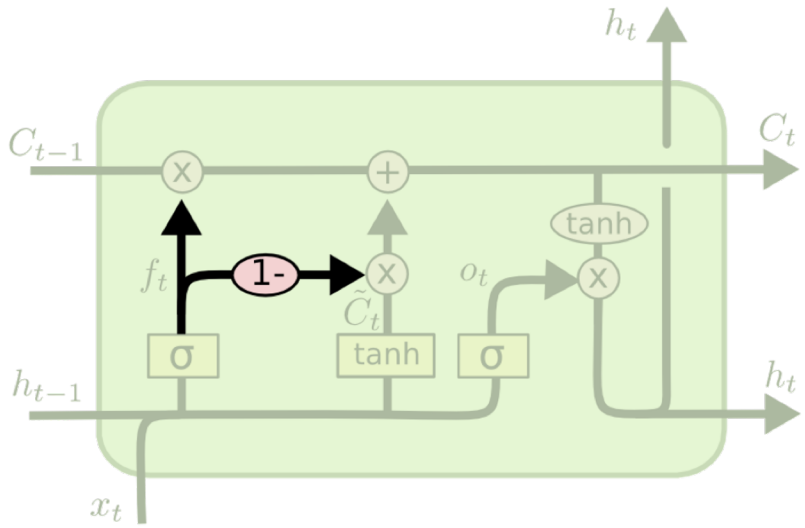
- Peephole connections
- Coupled Forget and Input Gates
- Gated Recurrent Unit
  - (Forget and Input Gates as single update gate)

Peephole LSTM



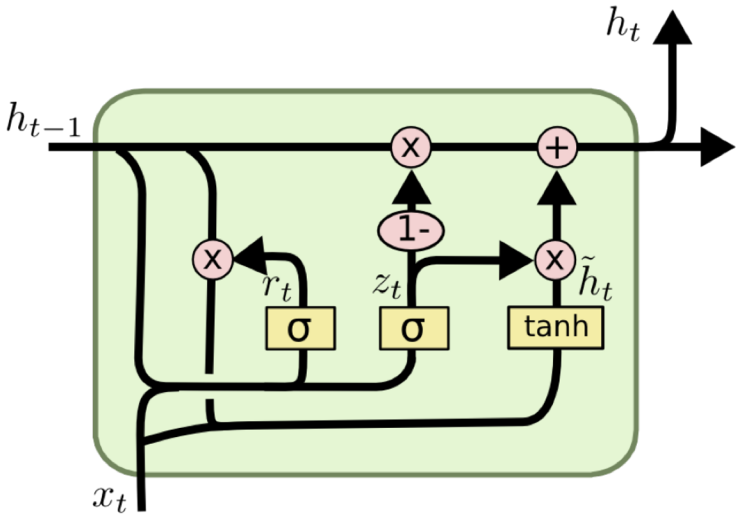
$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \; + \; b_o\right)$$

Coupled LSTM



$$C_t = f_t * C_{t-1} + (1 - \boldsymbol{f_t}) * \tilde{C}_t$$

GRU



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

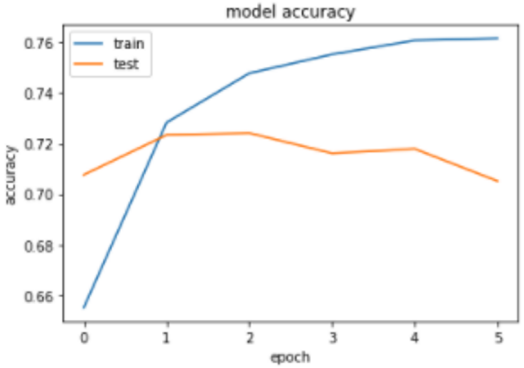
LSTM in Keras

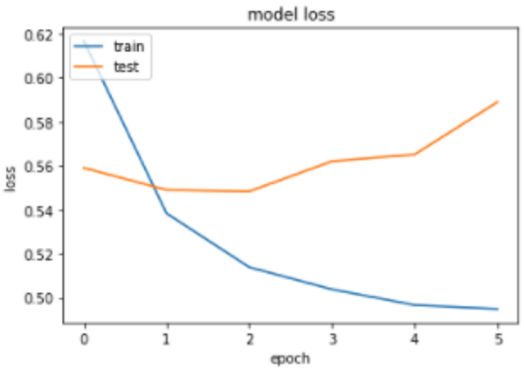
```
model = Sequential()
embedding_layer = Embedding(vocab_size,
                            50,
                            weights=[embedding_matrix],
                            input_length=maxlen,
                            trainable=False
)

model.add(embedding_layer)
model.add(LSTM(128))

model.add(Dense(1, activation='sigmoid'))
```

LSTM model summary

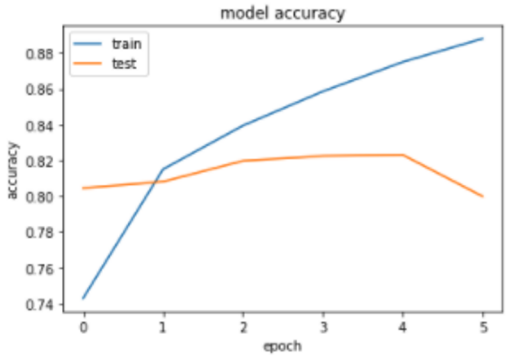
```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding (Embedding)        (None, 100, 50)           4627350
_________________________________________________________________
lstm (LSTM)                  (None, 128)               91648
_________________________________________________________________
dense (Dense)                (None, 1)                 129
=================================================================
Total params: 4,719,127
Trainable params: 91,777
Non-trainable params: 4,627,350
_________________________________________________________________
None
```

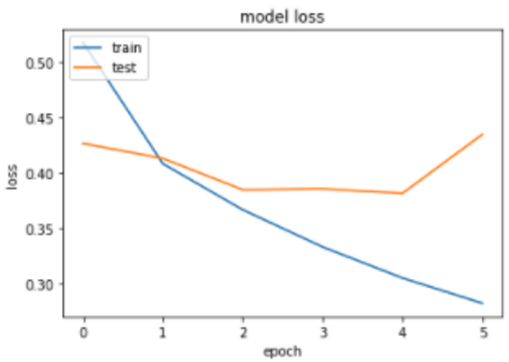
FCNN



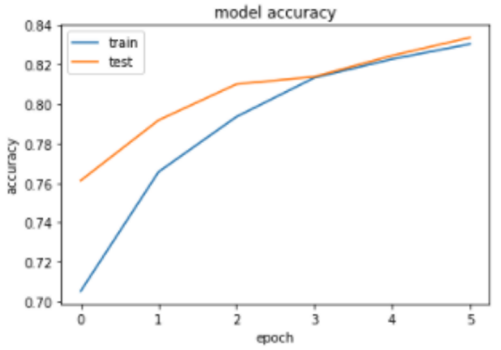


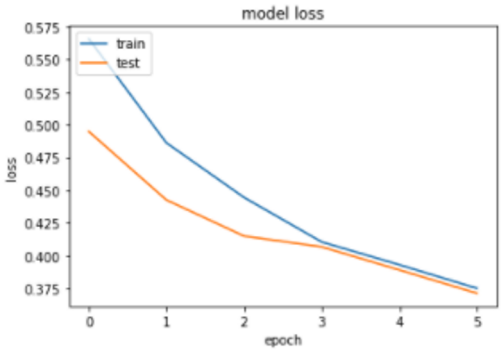
CNN





LSTM





Further Reading

Keras

https://keras.io/getting_started/intro_to_keras_for_engineers/

LSTM

http://karpathy.github.io/2015/05/21/rnn-effectiveness/

http://arxiv.org/pdf/1503.04069.pdf

http://jmlr.org/proceedings/papers/v37/jozefowicz15.pdf